# PR1 - Lesestoff zur Vorlesung 1

WS2026/27 (Stand 2026-08-31 15:56)

# Inhalt

L.1 Eine erste Einführung in Java ..... 2
L.1.1 Programmiersprachen ..... 2
L.1.2 Formulierung von Algorithmen ..... 3
L.1.2.1 Programmablaufpläne / Flussdiagramme ..... 4
L.1.2.2 Struktogramme/Nassi-Shneiderman-Diagramme ..... 4
L.1.2.3 Empfehlung ..... 4
L.1.3 Struktur von Java-Programmen ..... 4
L.1.3.1 Übungsaufgabe: Typische Programmierfehler ..... 4
L.1.3.2 Programmierkonventionen ..... 5
L.1.4 Details über Strings ..... 6
L.1.4.1 Spezielle Zeichen in Strings ..... 6
L.1.4.2 Übungsaufgabe ..... 6
L.1.4.3 Übungsaufgabe ..... 6
L.1.4.4 Mehrzeilige Strings ..... 6
L.1.5 Bezeichner und Schlüsselwörter ..... 8
L.1.5.1 Schlüsselwörter ..... 8
L.1.5.2 Sekundärtugenden für Programmiererinnen und Programmierer ..... 8
L.1.5.3 Wie tippt man ein Programm ein? ..... 11
L.1.6 Lösungen zu Übungsaufgaben ..... 16
L.1.7 Literatur ..... 16

# L.1 Eine erste Einführung in Java

## *L.1.1 Programmiersprachen*

Lesen Sie diesen Abschnitt im Kontext des Vorlesungsabschnitts 1.a

Eine Programmiersprache ist eine systematische Menge von Regeln zur Beschreibung von Anweisungen in einem durch Menschen bearbeitbaren Format. Wir werden die Programmiersprache Java lernen. Java wurde 1994/1995 entwickelt und fußt auf früheren einflussreichen Programmiersprachen.

Einige einflussreiche Programmiersprachen heben wir explizit hervor:

- FORTRAN: wurde für wissenschaftliches Rechnen konzipiert
- COBOL: Gegenentwurf für die Verarbeitung von Geschäftsdaten
- LISP: Zur Lösung von Problemen der Logik / Künstlichen Intelligenz
- BASIC: einfach zu erlernende Programmiersprache, die in der Ära der Heimcomputer Anfang der 1980er Jahre weit verbreitet war

Abbildung 1 zeigt die historische Abfolge weiterer Programmiersprachen und ihre gegenseitigen Einflüsse. Java erhielt wesentliche Ideen aus C++ und Smalltalk. Die Programmiersprache C++ und ihren Vorgänger C werden Sie in der Lehrveranstaltung Programmieren III im 3. Semester kennen lernen.

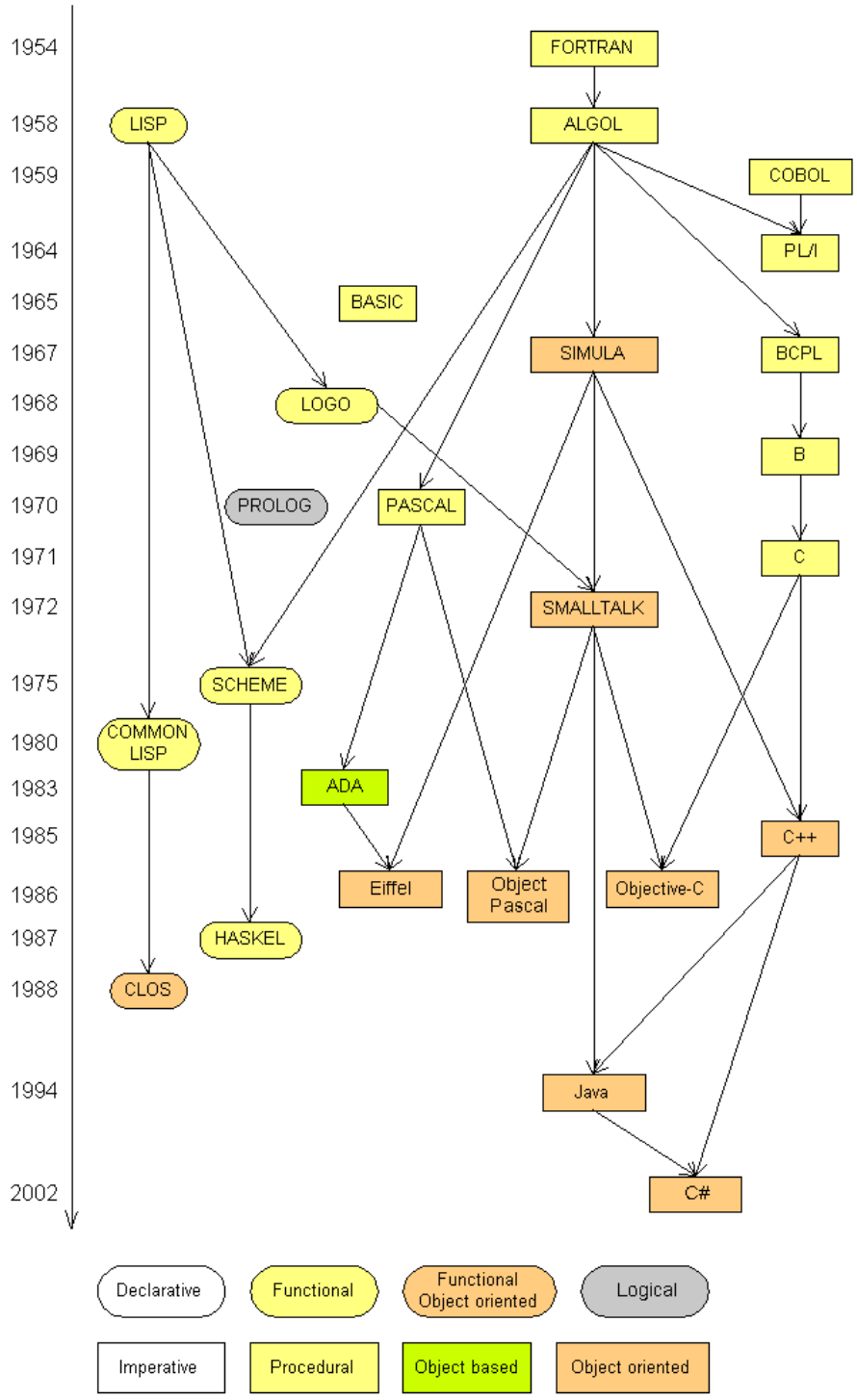



*Abbildung 1: Historie einiger Programmiersprachen (Quelle: [1])*

## *L.1.2 Formulierung von Algorithmen*

Lesen Sie diesen Abschnitt im Kontext des Vorlesungsabschnitts 1.b

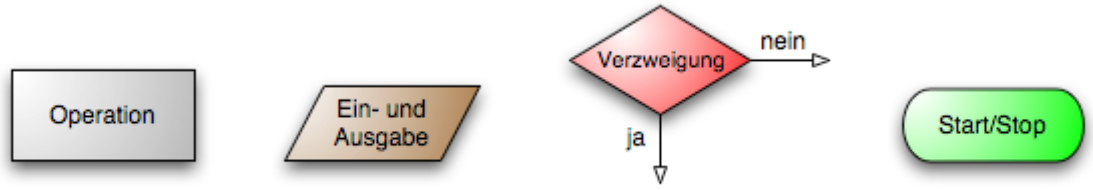
Ein Algorithmus ist eine Schritt-für-Schritt-Beschreibung zur Lösung eines Problems. Algorithmen werden von Menschen erdacht und werden daher in einer für Menschen verständlichen Sprache formuliert. Um Missverständnisse zu vermeiden, verwendet man häufig eine vorstrukturierte Form.

In der Vorlesung haben wir den Pseudocode als eine Formulierungsmöglichkeit kennen gelernt. Weitere Möglichkeiten setzen grafische Symbole zur Strukturierung ein.

### L.1.2.1 Programmablaufpläne / Flussdiagramme

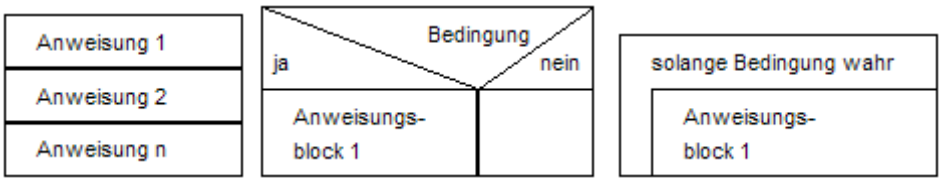
Programmablaufpläne (PAP) – oder synonym Flussidagramme – sind eine genormte Darstellungsform von Algorithmen. Sie entstanden (wie die nachfolgend genannten Struktogramme) in einer Zeit, als die Objektorientierung noch nicht erfunden war.

Im Folgenden stellen wir einige Symbole der Programmablaufpläne dar [2]:




### L.1.2.2 Struktogramme/Nassi-Shneiderman-Diagramme

Diagramme dieses Typs vermeiden Sprünge und sind damit einfacher in eine Programmiersprache ohne den GOTO-Befehl übersetzbar. Auch hier stellen wir einige Symbole vor [3]:




### L.1.2.3 Empfehlung

Beide Formen der grafischen Notation von Algorithmen (Programmablaufpläne und Struktogramme) sind in der Praxis heute kaum noch anzutreffen. Die UML (unified modelling language) hat mit dem Konzept des Aktivitätsdiagramms eine zeitgemäße Alternative parat. Die UML werden Sie im Verlaufe ihres Studiums noch detailliert kennen lernen. Für die Programmieren-Lehrveranstaltungen empfehlen wir die Formulierung von Algorithmen in Textform (Pseudocode).

## *L.1.3 Struktur von Java-Programmen*

Lesen Sie diesen Abschnitt im Kontext des Vorlesungsabschnitts 1.e

### L.1.3.1 Übungsaufgabe: Typische Programmierfehler

Betrachten Sie das folgende Programm und identifizieren Sie alle Programmierfehler:

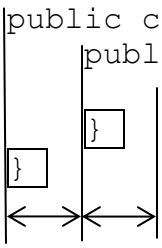
```
public MyProgram {
    public static main(string[] args {
        System.out.println("This is a test of the")
        System.out.println("emergency broadcast system.);
    }
}
```

Lösung: s. Abschnitt L.1.6.

### L.1.3.2 Programmierkonventionen

In den bisher präsentierten Programmen haben wir stets die öffnende geschweifte Klammer ans Ende der jeweiligen Programmzeile gesetzt. Die schließende geschweifte Klammer kam dann in eine eigene Programmzeile (sog. *Sun style*):

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, world!");
    }
}
```



je 4 Leerzeichen

Der Methodenkopf der `main`-Methode wurde um vier Leerzeichen eingerückt. Die Anweisung um weitere vier Leerzeichen. Die Bezeichnung des Programms (`Hello`) beschreibt die Funktion des Programms und beginnt mit einem Großbuchstaben. Festlegungen dieser Art nennt man **Programmierkonventionen**. Programmierkonventionen sind Vereinbarungen zwischen Menschen, um die Lesbarkeit von Programmen für Menschen zu verbessern. Dem Compiler sind Programmierkonventionen egal. Das obige Programm würde der Compiler auch wie folgt übersetzen können, die Lesbarkeit wäre jedoch stark eingeschränkt.

```
public class x { public static
void main(String[] args) { System.out.
println("Hello, world!");}}
```

Wir werden in den kommenden Kapiteln noch weitere Programmierkonventionen kennen lernen. Bei der Frage, wohin geschweifte Klammern gehören, gibt es zwei verbreitete Konventionen. Die erste haben Sie oben gesehen. Eine andere weit verbreitete Konvention sieht wie folgt aus (sog. *Allman style*):

```
public class Hello
{
    public static void main(String[] args)
    {
        System.out.println("Hello, world!");
    }
}
```

Hier wurden auch öffnende Klammern in eigenen Programmzeilen notiert.

Welche dieser beiden Konventionen Sie nutzen, ist hauptsächlich eine Geschmacksfrage. Wenn Sie sich für einen Stil entschieden haben, verwenden Sie ihn in ihren eigenen Projekten durchgängig. In dieser Lehrveranstaltung (Vorlesung und Übungen) verwenden wir durchgängig die erste Variante.

# *L.1.4 Details über Strings*

Lesen Sie diesen Abschnitt im Kontext des Vorlesungsabschnitts 1.f

## L.1.4.1 Spezielle Zeichen in Strings

Ein String (synonym: Zeichenkette) ist eine Folge von Zeichen, die ausgegeben oder vom Programm verarbeitet werden. Strings werden im Programmtext in Anführungszeichen eingefasst. Besonderheiten sind zu berücksichtigen, wenn spezielle Zeichen in den String aufgenommen werden sollen.

Ein String kann Spezialzeichen durch einen umgekehrten Schrägstrich (backslash) einleiten. Die Sequenz des backslash mit dem nachfolgenden Spezialzeichen nennt man auch **escape sequence**.

- **\t** Tabulator-Zeichen
- **\n** Zeichen für “neue Zeile”
- **\"** Anführungszeichen
- **** Backslash

Beispielsweise führt die Anweisung

```
System.out.println("\\Hallo\nwie\tgeht’s \"Dir\"?");
```

zur folgenden Ausgabe:

```
\Hallo
wie     geht’s "Dir"?
```

Ein beliebter Fehler ist die Verwendung des einfachen backslash in Dateipfaden:

```
System.out.println("C:\nin\the downward spiral");
```

führt zur Ausgabe

```
C:
in      he downward spiral
```

## L.1.4.2 Übungsaufgabe

Was geben die folgenden `println` Statements aus?

```
System.out.println("\ta\tb\tc");
System.out.println("\\\\");
System.out.println("'");
System.out.println("\"\"\"");
```

Lösung: s. Abschnitt L.1.6.

## L.1.4.3 Übungsaufgabe

Schreiben Sie ein `println` Statement für folgende Ausgabe:

```
/ \ // \\ /// \\\
```

Lösung: s. Abschnitt L.1.6.

## L.1.4.4 Mehrzeilige Strings

Will man mehrzeilige Strings ausgeben, kann man mehrere `println` Statements verwenden:

```
System.out.println("Liebe Dagmar,");
System.out.println();
System.out.println("ich lade dich ein zur Feier am");
System.out.println("  Mittwoch, 26.07.2023, 19 Uhr");
System.out.println("  ins \"LaSall\" in Hannover, Sallstr. 79");
System.out.println();
```

```
System.out.println("LG Fred");
```

In der Ausgabe werden Leerzeilen und Einrückungen ersichtlich:

```
Liebe Dagmar,

ich lade dich ein zur Feier am
  Mittwoch, 26.07.2023, 19 Uhr
  ins "LaSall" in Hannover, Sallstr. 79

LG Fred
```

Will man nicht mehrere `println` Statements einsetzen, konnte man bis Java 14 mit Standard-Methoden nur die folgende Variante verwenden, bei der ein String aus mehreren Teilstrings zusammengesetzt wird. Zeilenumbrüche muss man extra einfügen:

```
System.out.println("Liebe Dagmar,\n\n" +
   "ich lade dich ein zur Feier am\n" +
   "  Mittwoch, 26.07.2023, 19 Uhr\n" +
   "  ins \"LaSall\" in Hannover, Sallstr. 79\n\n" +
   "LG Fred");
```

Das ist fehleranfällig und unschön zu lesen, weil der ausgegebene Text mit zusätzlichen Anführungszeichen und Pluszeichen durchsetzt ist. Außerdem müssen die Anführungszeichen, die buchstäblich ausgegeben werden sollen, mit einem Backslash eingeleitet werden.

Ab Java 15 gibt es sog. **Text Blocks**, die die Aufgabe wesentlich vereinfachen:

```
System.out.println("""
    Liebe Dagmar,

    ich lade dich ein zur Feier am
      Mittwoch, 26.07.2023, 19 Uhr
      ins "LaSall" in Hannover, Sallstr. 79

    LG Fred
    """);
```

Die Einrückung des gesamten Textes um vier Leerzeichen ist hilfreich, um den Code zu strukturieren. Diese Einrückung wird vom Compiler bei der Bildung des Strings eliminiert. Nur die zusätzlichen zwei Leerzeichen Einrückung in der Mitte des Textes bleiben erhalten. Die Ausgabe dieses einen `println` Statements mit Text Block ist dieselbe wie beim ersten Programm, das aus mehreren `println` Statements besteht.

Einige Hinweise:

- Hinter den drei Anführungszeichen in der ersten Zeile darf nichts mehr stehen.
- Will man einen Backslash in den Text Block einfügen muss man ihn mit einem zusätzlichen Backslash einleiten.
- Weitere Besonderheiten, wie man Leerzeichen am Ende von einzelnen Zeilen erhalten kann, wie man Zeilenumbrüche in Text Blocks verhindert und wie man Platzhalter mit Werten befüllt, sind bspw. unter [5] nachlesbar.

## *L.1.5 Bezeichner und Schlüsselwörter*

Lesen Sie diesen Abschnitt im Kontext des Vorlesungsabschnitts 1.h

### L.1.5.1 Schlüsselwörter

Der Vollständigkeit halber nennen wir hier alle Schlüsselwörter in Java. Sie können keinen Bezeichner verwenden, der genauso wie eines der Schlüsselwörter heißt [6], [7]:

| | | | |
|---|---|---|---|
| `abstract` | `double` | `int` | `super` |
| `assert`*** | `else` | `interface` | `switch` |
| `boolean` | `enum`**** | `long` | `synchronized` |
| `break` | `extends` | `native` | `this` |
| `byte` | `final` | `new` | `throw` |
| `case` | `finally` | `package` | `throws` |
| `catch` | `float` | `private` | `transient` |
| `char` | `for` | `protected` | `try` |
| `class` | `goto`* | `public` | `void` |
| `const`* | `if` | `return` | `volatile` |
| `continue` | `implements` | `short` | `while` |
| `default` | `import` | `static` | `_ (underscore)`***** |
| `do` | `instanceof` | `strictfp`** | |

| | |
|---|---|
| * | not used |
| ** | added in 1.2 |
| *** | added in 1.4 |
| **** | added in 5.0 |
| ***** | added in 9.0 |

`const` und `goto` sind reserviert – werden jedoch derzeit in Java nicht verwendet. Auch `strictfp` besitzt als Schlüsselwort keine Verwendung mehr.

Über die Schlüsselwörter hinaus können Sie folgende Bezeichner nicht verwenden, weil es sich um Java-Literale handelt: `true`, `false` und `null` .

Seit Java 10 wird `var` als Bezeichner mit einer besonderen Bedeutung genutzt. Als Bezeichner für einen Typnamen (Klassennamen) ist `var` seit Java 10 nicht mehr erlaubt.

### L.1.5.2 Sekundärtugenden für Programmiererinnen und Programmierer

Wir zitieren zunächst aus einem Wikipedia-Artikel [4] über die sogenannten Sekundärtugenden:

> *Als Sekundärtugenden oder auch bürgerliche Tugenden wurden Charaktereigenschaften eingestuft, die zur praktischen Bewältigung des Alltags und zum „störungsfreien“ Betrieb einer Gesellschaft beitrügen, ohne aber für sich allein eine ethische Bedeutung zu haben, sofern sie als Selbstzweck hochgehalten würden und nicht zur Umsetzung der Primärtugenden dienten.*
>
> *Zu den bürgerlichen oder Sekundärtugenden wurden insbesondere Fleiß, Treue, Gehorsam, Disziplin, Pflichtbewusstsein, Pünktlichkeit, Zuverlässigkeit, Ordnungsliebe, Höflichkeit, Sauberkeit u. a. m. gezählt, meist aus dem Katalog der preußischen Tugenden bzw. des „bürgerlichen“ Tugendkatalogs. […]*

> *Nach 1968 (Studentenbewegung) gerieten die bürgerlichen Werte in die Kritik und wurden abgewertet oder verächtlich gemacht […]. Stattdessen wurden postmaterialistische Werte wie Menschlichkeit, Kreativität, Selbstverwirklichung und soziale Werte wie Solidarität betont.*

Sie werden sich vermutlich fragen: Was hat dieser Artikel mit Programmieren zu tun?

Ich beobachte regelmäßig Programmieranfänger, die sich schwer tun, syntaktisch korrekte Programme zu schreiben. Häufig werden Klammern vergessen oder an einer falschen Stelle eingefügt. Anführungszeichen zur Begrenzung eines auf der Console auszugebenden Textes fehlen. Ein Semikolon wird nicht oder an der falschen Stelle eingefügt. Bezeichner werden beliebig gewählt. Die Liste ließe sich mit weiteren Syntaxverstößen fortführen. Ein Beispiel:

```
01 public class hello {
02   public static void Main(String[] args)
03   System.out.println("Guten tag!  );
04       System.out.println  "Auf Wiedersehen"
05 }
06 )
07
08 public static void methode {
09       System.out.printn  (  "Hallo  ");
10
11       public static methode2 () {
12         System.out.println("Welt");
13         }
14     }
15 }
```

Ein schlampig erstelltes Programm

Das vorstehende Programm wurde schlampig erstellt. Bei der Programmierung mangelte es an Disziplin, Ordnungsliebe und Sauberkeit (um Begriffe des obigen Wikipedia-Artikels aufzugreifen).

Ich möchte dieses Beispiel nutzen, um die Fehler zu illustrieren, die sich einschleichen, wenn man als Programmierer oder Programmiererin keine Disziplin, Ordnungsliebe und Sauberkeit walten lässt.

- Die Programmierkonvention „Klassennamen beginnen mit einem Großbuchstaben“ wurde ignoriert.
- Die Regel „bei Bezeichnern in Java kommt es auf Groß-/Kleinschreibung an“ wurde bei der Programmierung der Methode Main ignoriert.
- Die Programmierkonventionen „Methodenköpfe werden 4 Leerzeichen eingerückt“ und „Statements werden weitere 4 Leerzeichen eingerückt“ wurden ignoriert.
- Die Syntaxregel „Jede Anweisung endet mit einem Semikolon“ wurde ignoriert.
- Die Syntaxregel „Jede Methode beginnt mit den Schlüsselwörtern `public static void`“ wurde ignoriert.
- Die Syntaxregel „Jedem einleitenden Anführungszeichen zu Beginn eines Textes muss ein Anführungszeichen am Ende des Textes folgen“ wurde ignoriert.
- Die Syntaxregel „Auszugebende Texte müssen in Anführungszeichen eingefasst werden“ wurde ignoriert.
- Die Syntaxregel „Ausgabeanweisungen mit `System.out.println` werden mit zwei runden Klammern `()` und ggf. einem Ausgabetext dazwischen formuliert“ wurde ignoriert.

- Die Regel „Bezeichner müssen exakt eingegeben werden“ wurde z. B. in Zeile 09 ignoriert (`printn` statt `println`).

Alle diese Fehler hätten recht leicht vermieden werden können, wenn sich die Programmiererin oder der Programmierer diszipliniert an die zugegebenermaßen vielen Regeln gehalten hätte, die jede für sich nicht schwer zu verstehen ist. Die Gesamtheit aller zu berücksichtigenden Regeln macht dies manchmal schwer. Dennoch hilft es ungemein, diszipliniert und ordentlich vorzugehen und einen sauberen und aufgeräumten Programmcode zu schreiben.

Manche Studierende hoffen darauf, dass der Compiler ihnen ihre Fehler schon mitteilen wird. Dies ist häufig ein Wunsch, der sich nicht erfüllt. Der Compiler spuckt zum obigen Programm folgende Meldungen aus:

```
$ javac Hello.java
Hello.java:2: error: ';' expected
  public static void Main(String[] args)
                                        ^
Hello.java:3: error: unclosed string literal
  System.out.println("Guten tag!  );
                     ^
Hello.java:6: error: class, interface, or enum expected
)
^
Hello.java:8: error: class, interface, or enum expected
public static void methode {
              ^
Hello.java:11: error: class, interface, or enum expected
      public static methode2 () {
                    ^
Hello.java:13: error: class, interface, or enum expected
        }
        ^
6 errors
```

Die erste Fehlermeldung, dass ein Semikolon fehlt, führt die besagten Studierenden schnurstracks in die Irre und das Resultat ist noch fehlerhafter als vorher ☹.

Ich möchte Ihnen, wenn Sie zu denjenigen Studierenden gehören, die schon einmal ein vor Fehlern überbordendes Programm wie das obige geschrieben haben, raten, ein paar einfache Merksätze zu Herzen zu nehmen:

- Legen Sie sich einen persönlichen Regelkatalog an. Damit meine ich ein DIN A4-Blatt, auf dem Sie die Regeln auflisten.
- Wenn Sie eine Zeile Code geschrieben haben, prüfen Sie sofort (bevor Sie weiter programmieren), ob diese Zeile allen Regeln in Ihrem Regelkatalog gehorcht.
- Tun Sie dies konsequent und diszipliniert. Gehen Sie gewissenhaft jede einzelne Regel durch und halten Ihre Programmzeile daneben. Wenn Ihre Programmzeile einer Regel nicht gehorcht, korrigieren Sie sie erst, bevor Sie weiter machen. Hoffen Sie nicht auf einen klugen Compiler, der Ihnen diese mühevolle Arbeit abnimmt. Einen solchen Compiler gibt es nicht!
- Verwenden Sie den Editor jEdit (nicht gedit !) und achten regelmäßig darauf, dass Sie die Datei im richtigen Editor geöffnet haben.

- Wenn Sie Sicherungskopien von Quelldateien machen, halten Sie Ordnung. Sonst passiert es Ihnen, dass Sie den Quelltext in `~/sicherungen/Hello.java` ändern, aber den Quelltext in `~/ueb1/Hello.java` übersetzen und ausführen. Und Sie wundern sich, warum Ihre Codeänderungen keinen Effekt haben …
- Wenn Sie eine öffnende Klammer eintippen schreiben Sie sofort danach die zugehörige schließende Klammer.
- Wenn Sie ein Statement schreiben, ergänzen Sie sofort das Semikolon am Ende
- Wenn Sie ein Anführungszeichen eingeben, ergänzen Sie sofort das zugehörige abschließende Anführungszeichen.

Wenn Sie diese Merksätze diszipliniert verfolgen, schließen Sie viele Fehlerquellen von vorneherein aus. In diesem Sinne halte ich Disziplin für eine wichtige Sekundärtugend für Programmiererinnen und Programmierer.

Interessanterweise kommt es in der Software-Entwicklung nicht nur auf die klassischen Tugenden (Disziplin, Ordnungsliebe, Sauberkeit) an, sondern auch auf die im obigen Wikipedia-Artikel anklingenden Werte Menschlichkeit, Kreativität, Selbstverwirklichung, Solidarität. Letztere werden dann wichtig, wenn Sie Software in einem interdisziplinären Team erstellen und dabei innovative Funktionen für sich rasch ändernde Anforderungen und Märkte für ein weites Spektrum von nutzenden Menschen und Organisationen realisieren.

### L.1.5.3 Wie tippt man ein Programm ein?

Eine Beispielsequenz soll illustrieren, wie Sie sich schon beim Eingeben eines Programms das Leben leichter machen können, indem Sie viele Fehlerquellen von vorneherein ausschließen.

1. Sie tippen:

```
1 public class Hello {
```

Der Eingabecursor (hier dargestellt als | ) blinkt direkt hinter der geöffneten geschweiften Klammer. Bevor Sie nun irgendetwas anderes eingeben, speichern Sie die Datei unter dem Namen `Klassenname.java` (also hier `Hello.java`). Danach sieht Ihr Programm so aus:

```
1 public class Hello {
```

Bevor Sie nun irgendetwas anderes eingeben, ergänzen Sie eine schließende geschweifte Klammer genau unterhalb des `p` von `public`. Dazu betätigen Sie zwei Mal RETURN und dann einmal die schließende geschweifte Klammer:

```
1 public class Hello {
2
3 }
```

Nachdem Sie die Klammer eingefügt haben, positionieren Sie den Cursor ins „Innere“ der Klasse und zwar genau 4 Leerzeichen eingerückt:

```
public class Hello {

}
```

Hier fahren Sie nun fort.

2. Sie tippen:

```
public class Hello {
    public static void main(
}
```

Bevor Sie `String[] args` eingeben, schließen Sie zuerst die runde Klammer:

```
public class Hello {
    public static void main()
}
```

Dann positionieren Sie den Cursor zwischen die Klammern;

```
public class Hello {
    public static void main()
}
```

Und nun tippen Sie weiter:

```
public class Hello {
    public static void main(String[] args)
}
```

3. Sie tippen eine öffnende geschweifte Klammer:

```
public class Hello {
    public static void main(String[] args) {
}
```

Betätigen Sie nun zwei Mal RETURN und ergänzen sofort die zugehörige schließende geschweifte Klammer:

```
public class Hello {
    public static void main(String[] args) {

    }
}
```

Die schließende Klammer landet genau an der richtigen Stelle, wenn Sie die Datei unter einem Dateinamen Hello.java abgespeichert haben (auf die Dateinamensendung java kommt es an).
Positionieren Sie nun den Cursor ins Innere der Methode und zwar genau 8 Leerzeichen eingerückt:

```
public class Hello {
    public static void main(String[] args) {

    }
}
```

Hier fahren Sie nun fort.

4. Sie tippen den Anfang einer Ausgabeanweisung:

```
public class Hello {
    public static void main(String[] args) {
        System.out.println(
    }
}
```

Schließen Sie nun sofort die runde Klammer und ergänzen sofort das Semikolon am Ende der Anweisung:

```
public class Hello {
    public static void main(String[] args) {
        System.out.println();
    }
}
```

Positionieren Sie nun den Cursor genau zwischen die beiden runden Klammern:

```
public class Hello {
    public static void main(String[] args) {
        System.out.println();
    }
}
```

Hier fahren Sie nun fort mit dem auszugebenden Text

5. Beginnen Sie den auszugebenden Text mit einem Anführungszeichen:

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("");
    }
}
```

Ergänzen Sie nun sofort das zugehörige Ende-Anführungszeichen:

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("");
    }
}
```

Positionieren Sie nun den Cursor zwischen die Anführungszeichen:

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("");
    }
}
```

Und hier tippen Sie nun weiter:

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello world!");
    }
}
```

6. Sie wollen eine zweite Ausgabeanweisung ergänzen. Positionieren Sie dazu den Cursor hinter das Semikolon der ersten Anweisung:

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello world!");
    }
}
```

Nun betätigen Sie einmal RETURN:

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello world!");

    }
}
```

Der Cursor landet genau an der richtigen Stelle unterhalb der vorherigen Anweisung. Beginnen Sie hier nun mit dem Tippen der zweiten Anweisung:

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello world!");
        Syst...
    }
}
```

7. Sie wollen eine weitere Methode ergänzen. Positionieren Sie dazu den Cursor rechts neben die geschweifte Klammer, die die vorherige Methode abschließt:

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello world!");
        System.out.println("And good bye");
    }
}
```

Betätigen Sie nun einmal oder zweimal RETURN:

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello world!");
        System.out.println("And good bye");
    }


}
```

Der Cursor landet genau an der richtigen Position. Tippen Sie nun den Beginn der zweiten Methode:

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello world!");
        System.out.println("And good bye");
    }

    public static void ...
}
```

## L.1.6 Lösungen zu Übungsaufgaben

Lösung zu Übungsaufgabe L.1.3.1:
Die sechs Fehler sind im folgenden Programm hervor gehoben:

```
public class MyProgram {
    public static void main(String[] args) {
        System.out.println("This is a test of the");
        System.out.println("emergency broadcast system.");
    }
}
```

Lösung zu Übungsaufgabe L.1.4.2:

```
    a     b     c
\\
'
""""
```

Lösung zu Übungsaufgabe L.1.4.3:

```
System.out.println("/ \\ // \\\\ /// \\\\\\");
```

## L.1.7 Literatur


[1] Verkeyn, Andy: History of Programming Languages. URL: http://www.verkeyn.net/professional/histlang.html, Version vom 13.04.2015, zuletzt abgerufen am 22.08.2019.

[2] Wikipedia-Autoren: Programmablaufplan. URL: https://de.wikipedia.org/w/index.php?title=Programmablaufplan&oldid=191154626, Version vom 8.08.2019

[3] Wikipedia-Autoren: Nassi-Shneiderman-Diagramm. URL: https://de.wikipedia.org/w/index.php?title=Nassi-Shneiderman-Diagramm&oldid=188900306, Version vom 24.05.2019

[4] Wikipedia-Autoren: Sekundärtugend. URL: https://de.wikipedia.org/w/index.php?title=Sekund%C3%A4rtugend&oldid=144478596, Version vom 28.07.2015

[5] Ullenboom, Christian: Insel: Textblöcke (text blocks). URL: http://www.tutego.de/blog/javainsel/2020/09/insel-textbloecke-text-blocks/, Version vom 25.09.2020

[6] o. V.: Java Language Keywords. URL: http://docs.oracle.com/javase/tutorial/java/nutsandbolts/_keywords.html, zuletzt abgerufen am 22.08.2019.

[7] Gosling, James; Joy, Billy; Steele, Guy; Bracha, Gilad; Buckley, Alex; Smith, Daniel: The Java Language Specification – Java SE 10 Edition, Abschnitt 3.9. URL: https://docs.oracle.com/javase/specs/jls/se10/html/jls-3.html#jls-3.9, zuletzt abgerufen am 22.08.2019.